MINISTRY OF COMMUNICATIONS & INFORMATION TECHNOLOGY (MCIT)
DIGITAL EGYPT PIONEERS INITIATIVE (DEPI)

PORTFOLIOGENIE

An AI-Powered Developer Portfolio Compiler and Static Exporter

GRADUATION PROJECT DOCUMENTATION

Prepared by Team Members:

Elsayed Mohamed Elsayed Mohamed Farag
Mohamed Sherif Abdelrahim
Mostafa Mohamed Taha Ali Mohamed
Youssef Shady Gabr Abdo
Youssef Mohamed Mohamed Sadeq
Ahmed Mostafa Riad Mostafa

Table of Contents

List of Figures

List of Tables

1. Project Planning & Management

1.1 Project Introduction & Proposal Overview

1.1.1 Introduction

In the modern digital era, the professional identity of software developers has shifted from static, paper-based resumes to dynamic, online portfolios. A developer's portfolio is not merely a list of credentials, but a live showcase of their engineering capabilities, code quality, design sensibilities, and technical expertise. As recruiters and engineering managers increasingly rely on GitHub profiles and interactive demonstrations to evaluate candidates, having a polished, accessible, and fast web presence is critical for job seekers.

However, building a custom portfolio from scratch presents significant friction. It requires developers to spend hours configuring layout stylesheets, writing copy, managing API requests, and deploying files on cloud hosting services. PortfolioGenie addresses this challenge by providing a zero-configuration, client-side portfolio compiler. It empowers developers to generate a high-performance, single-page portfolio in under five minutes, letting them focus their energy on core programming and problem-solving.

1.1.2 Overview of the Project

In the contemporary software engineering employment landscape, a developer's professional portfolio serves as their primary currency. Traditional PDF resumes lack the interactive capacity to demonstrate coding style, repository health, web performance, and responsive layout adaptivity. Employers and recruiters seek accessible proof of skills rather than self-reported statements. However, junior web developers, students, and active job seekers often suffer from the 'builder's block'—they spend valuable days struggling to format raw CSS sheets, writing repetitive descriptions, manually syncing project links, and fighting configurations to host their portfolios. This process deflects their focus from practicing core algorithms and backend software design.

PortfolioGenie was conceived to solve this critical problem. Developed as a graduation project under the Full-Stack React Web Development Track of the Digital Egypt Pioneers Initiative (DEPI) sponsored by the Ministry of Communications and Information Technology (MCIT), PortfolioGenie is a client-side Single Page Application (SPA) designed to automate the developer portfolio building lifecycle. The application connects directly with the GitHub API. By simply inputting a GitHub handle, the platform extracts profile information, retrieves public repositories metadata, parses language usage statistics to aggregate developer skills, and uses structured simulated AI copywriting models to refine biographies and project summaries. Furthermore, the system provides a side-by-side live interactive preview where changes to profile fields, social links, work experience, and educational background are synced in real time. The absolute novelty of PortfolioGenie is its single-page compiler utility, which packages all local CSS styles, assets, SVG icons, and reactive DOM templates into a single standalone static index.html file that the user can download. This exported file is entirely portable, zero-dependency, and ready to be hosted on free CDNs (such as Netlify, Vercel, or GitHub Pages) without incurring hosting costs or platform lock-in constraints.

1.1.3 Project Objectives

To validate the project's engineering quality, the development team set clear, measurable, and realistic goals:

1. Automate Portfolio Development: Reduce the time required to build a responsive portfolio from an average of 16 developer hours to less than 5 minutes.
2. GitHub API Integration: Establish a reliable synchronization system that fetches public repository names, stars, descriptions, and programming languages without requiring user credentials.
3. AI Copywriting Enhancements: Implement simulated technical copywriting heuristics using contextual prompt templates to elevate basic user inputs into high-impact, professional bios.
4. Single-Page Portable Compilation: Build a client-side exporter capable of serializing CSS styles, SVG assets, and dynamic layouts into a single, light-weight HTML file.
5. Responsive Design Performance: Create a flexible design that renders correctly across mobile (320px) to ultra-wide (1920px) screen viewports.
6. Serverless Local Caching: Avoid database hosting costs and preserve user privacy by using LocalStorage to persist user state.

1.1.4 Project Scope

The boundaries of PortfolioGenie were clearly defined to ensure focused execution within the initiative's timeline:

In-Scope Features:

- Landing Page: Introduces features, benefits, dashboard mockups, and entry points.
- Setup Wizard/Editor: 3-step dashboard for Profile (including avatar upload and social links), Projects (fetched or manual), and Experience/Education details.
- GitHub API Hook: Fetches user metadata, bio, avatar, public repositories, and language statistics.
- AI Text Enhancement Hook: Context-based simulated AI rewriting for bio and project descriptions.
- Real-Time Canvas: Renders a preview of the portfolio page showing changes in real time.
- Static HTML Exporter: Utility compiling stylesheets and layouts into a single downloadable file.
- Local Persistence: Caches user input automatically in browser local storage.

Out-of-Scope Features:

- Backend User Authentication: No signup, login, or password systems (the project is fully client-side to ensure serverless portability).
- Database Hosting: No cloud database storage (all persistence is handled locally on the client's machine).
- Custom Domain registrar: The platform does not purchase or host domain names for users.
- Dynamic Server Side Rendering (SSR): The application does not require backend servers to render portfolios.

1.2 Project Plan

1.2.1 Timeline, Gantt Chart and Progress Status

The project was executed over a span of 20 weeks starting in February 2026. The schedule was structured to balance research, design, coding, testing, and deployment. The table below represents the detailed project tasks, timeline durations, and deliverables:

Table 1.1: Detailed Project Gantt Tasks Schedule

Task ID	Task Name	Duration	Start Date	End Date	Task Status
T1	Project Setup & Planning	12 Days	Feb 3, 2026	Feb 15, 2026	Completed
T2	Literature & Technology Review	14 Days	Feb 16, 2026	Mar 1, 2026	Completed
T3	Requirements Gathering & SRS Draft	19 Days	Mar 2, 2026	Mar 20, 2026	Completed
T4	UML System Modeling (DFD, ERD, Use Cases)	26 Days	Mar 21, 2026	Apr 15, 2026	Completed
T5	Vite+React SPA Skeleton Setup & Routing	20 Days	Apr 16, 2026	May 5, 2026	Completed
T6	GitHub REST API Hook Integration	20 Days	May 6, 2026	May 25, 2026	Completed
T7	AI Generation & Prompt Optimization	16 Days	May 26, 2026	Jun 10, 2026	Completed
T8	Static Compiler & HTML Exporter Utility	15 Days	Jun 11, 2026	Jun 25, 2026	Completed
T9	UI/UX Responsiveness & Glassmorphism Polish	13 Days	Jun 26, 2026	Jul 8, 2026	Completed

T10	System Manual Testing & Quality Assurance	5 Days	Jul 9, 2026	Jul 13, 2026	Completed
T11	Documentation Compilation & docx build	2 Days	Jul 14, 2026	Jul 15, 2026	Completed
T12	Graduation Presentation & Video Demo	2 Days	Jul 16, 2026	Jul 17, 2026	Completed

1.2.2 Key Project Milestones

Milestones mark the completion of major phases and serve as checkpoints for the MCIT review board:

- Milestone 1: Project Proposal Approval - Feb 15, 2026 (Scope defined and proposal accepted).
- Milestone 2: Requirements Specification Signed - Mar 20, 2026 (SRS signed off by advisors).
- Milestone 3: System Design & UML Freeze - Apr 15, 2026 (All architectural diagrams approved).
- Milestone 4: Code Integration Completed - Jun 25, 2026 (Vite components, API hooks, and Exporter combined).
- Milestone 5: System Testing Finished - Jul 13, 2026 (Zero critical bugs reported in testing suites).
- Milestone 6: Final Submission & Handover - Jul 17, 2026 (Documentation, code, slides, and video demo uploaded to GitHub).

1.2.3 Core Project Deliverables

The team commits to delivering the following artifacts upon graduation:

1. Source Code Repository: A public GitHub repository containing clean, documented React code, dependencies, and Vite configurations.
2. Technical Project Documentation: An academic report (this document) describing planning, architecture, schemas, and test results.
3. Deployment Link: A live production URL hosted on Vercel.
4. Presentation Slide Deck: A PDF summarising the project architecture, design, DFD, and test results.
5. Video Demonstration: A 5-minute video walkthrough of the app's features.

1.2.4 Resource Allocation & Infrastructure

The project was executed using the following resources:

- Human Resources: A team of 6 full-stack developer students from DEPI.
- Software Stack: Node.js, React 19, Vite, Lucide React, Framer Motion, HTML5, CSS3, Vercel CDN, and Git/GitHub.
- Development Tools: VS Code, browser developer tools (Lighthouse), UML modeling tools (Mermaid), and Microsoft Word for compilation.
- Hardware Resources: Laptops with standard configurations and a stable internet connection for API requests.

1.3 Task Assignment & Team Roles

1.3.1 Definition of Team Roles

To ensure clear coordination, team members were assigned specific roles based on their interests and strengths. Below is the task allocation table showing roles and files owned in the repository:

Table 1.2: Task Assignment & Team Roles

Team Member Name	Technical Role	Files Owned	Main Assigned Tasks
Elsayed Mohamed Elsayed Mohamed Farag	Team Lead, UI/UX Designer & Lead Technical Writer	Project planning and coordination (Scrum Master), visual identity design system, landing page component implementation, and lead author for project technical documentation.	Architecture design, Landing page routing, React context provider and state management.
Mohamed Sherif Abdelrahim	Profile Builder & Canvas Layout Developer	Implementing the profile edit form (ProfileTab), client-side Canvas base64 profile picture uploader, and the live portfolio preview canvas component (PortfolioCanvas.jsx).	Designing CSS classes, animations, media queries, and responsive layouts.
Mostafa Mohamed Taha Ali Mohamed	GitHub REST API Sync Hook Developer	Implementing the custom GitHub API fetching hook (useGithub.js), public repositories metadata parser, automatic language statistics extraction, and client-side rate limit caching.	Fetching public repositories, mapping metadata, and extracting skills tags.

Youssef Shady Gabr Abdo	Projects Customizer & Editor Developer	Implementing the projects tab components, allowing users to customize repository names/descriptions, manage programming language badges, and append custom projects manually.	Designing prompt templates, bio simulation, and copywriting enhancement logic.
Youssef Mohamed Mohamed Sadeq	Experience & Education Timeline Developer	Implementing dynamic timeline builder forms, allowing users to add, customize, and delete job experience cards and education history cards.	Serializing DOM templates, embedding styles, and file download mechanisms.
Ahmed Mostafa Riad Mostafa	AI Enhancer & Static Exporter Developer	Implementing the simulated AI copywriting enhancer hook (useAI.js), global state context (PortfolioContext.jsx), localStorage autosave sync, and the static HTML bundle compiler exporter utility (exportPortfolio.js).	Manual testing execution, bug reporting, and technical writing.

1.3.2 Detailed Responsibilities and Module Ownership

1. Elsayed Mohamed Elsayed Mohamed Farag (Team Lead & System Architect):
Elsayed was responsible for defining the system architecture and state flow. He initialized the Vite project template, configured React Router Dom, and built the main route nodes (LandingPage and Builder). He authored the global state provider, PortfolioContext.jsx, which handles profile details, repositories, experience, and education updates. He also managed the Vercel deployment and configured vercel.json for fallback routes.

2. Mohamed Sherif Abdelrahim (Lead UI/UX Developer):
Mohamed Sherif owned the UI/UX design of the application. He created the dark mode color scheme (#0B0F19, #00D4FF, #A855F7), implemented glassmorphism styling, and handled media queries for responsiveness. He integrated Framer Motion to create smooth animations when transitioning between wizard steps and loading elements.

3. Mostafa Mohamed Taha Ali Mohamed (API & Middleware Integrator):
Mostafa was responsible for integrating external APIs. He developed usePortfolioHooks.js and mapped the GitHub REST API endpoints to retrieve profile data, bio summaries, and public repositories. He also wrote the logic to extract programming languages and compile them into skill tags.

4. Youssef Shady Gabr Abdo (AI Services Developer):
Youssef Shady led the AI text enhancement implementation. He designed the prompt structures in aiService.js to format user inputs into structured prompts. He also built the client-side simulation logic, providing realistic and fast response templates for the AI bio enhancer.

5. Youssef Mohamed Mohamed Sadeq (Static Exporter Developer):
Youssef Sadeq developed the HTML compiler. He authored exportPortfolio.js, which reads the current state from PortfolioContext, styles it, and compiles it into a single-file static HTML structure. He also built the browser-side file downloader that saves the compiled HTML file to the user's computer.

6. Ahmed Mostafa Riad Mostafa (QA Lead & Technical Writer):
Ahmed directed the quality assurance phase. He wrote the manual test suites and evaluated the application on different browsers (Chrome, Edge, Firefox, Safari) and screen sizes. He logged bugs in the issue tracker and led the documentation compilation process, preparing the final Word document.

1.4 Risk Assessment & Mitigation Plan

To avoid project delays, the team evaluated potential technical, operational, and external risks, establishing the following mitigation plans:

Table 1.3: Risk Assessment and Mitigation Fallback Strategies

Risk ID	Risk Description	Probability	Impact	Mitigation & Fallback Plan
R-01	GitHub API rate limits on unauthorized calls (60 requests/hr limit).	High	Medium	Cache responses in browser LocalStorage; notify users to use personal access tokens if limits are exceeded.
R-02	AI service downtime or API token costs during evaluation.	Medium	High	Design prompt strings locally, compile using a mock simulation client-side to ensure zero-cost testing.
R-03	Browser storage limits (LocalStorage capacity is limited to 5MB).	Low	Medium	Compress profile avatar upload images via canvas conversion to base64, storing only text metadata.
R-04	Vercel routing returning 404 when reloading sub-pages.	Medium	Low	Create vercel.json configuration redirecting all URL paths to index.html.
R-05	Layout breakage of exported portfolio HTML across older browsers.	Low	High	Use standard CSS variables and basic inline styles without complex flex/grid experimental properties.

R-06	Merge conflicts on GitHub during parallel development.	Medium	Medium	Establish Git branch guidelines, enforce pull request reviews, and use a dedicated dev branch.
R-07	Project scope creep due to additional feature ideas.	Low	High	Stick to defined goals and milestones, deferring extra features to future releases.

1.5 Key Performance Indicators (KPIs)

To measure the technical success and user satisfaction of the system, we tracked the following key performance metrics:

Table 1.4: Key Performance Indicators Success Metrics

KPI ID	Metric Name	Target success criteria	Measurement Method
KPI-01	Application Load Speed	Lighthouse Performance Score >= 90 (Load < 1.5s)	Lighthouse test tools on compiled production builds.
KPI-02	Data Sync Latency	LocalStorage write delay < 500ms after input	Performance profiling in browser developer tools.
KPI-03	GitHub Fetch Speed	Complete repository mapping completed < 4 seconds	Console time profiling for Fetch API requests.
KPI-04	HTML Export Integrity	100% style matching and zero external file requirements	Manual layout checks on exported files.

KPI-05	UI Responsive Scale	Smooth layout scaling across mobile (320px) to ultra-wide (1920px)	Mobile viewport simulation testing in QA.
KPI-06	AI Simulation Latency	AI bio generator response returned within 1.5 seconds	Debounce time monitoring in code hooks.

2. Literature Review

2.1 Feedback & Evaluation of Existing Literature

To establish the foundation of PortfolioGenie, the team conducted a detailed literature review of existing profile builders, evaluating platforms like Wix, Squarespace, standard HTML templates, and developer-specific profile tools (e.g., GitHub Readme Generators). During early presentations to MCIT academic advisors and track supervisors, the project received valuable feedback that shaped the system requirements:

1. API Rate Limiting Warnings: Track supervisors warned that unauthorized client-side calls to public GitHub endpoints are limited to 60 requests per hour per IP. If a user repeatedly updates their profile, the app could return rate limit errors. The evaluation committee recommended caching the API response in browser localStorage to minimize API calls.

2. Security of AI API Keys: Advisors highlighted that integrating a live LLM (like Gemini or OpenAI) on a client-side SPA raises security concerns, as the API key would be exposed in the frontend code. They suggested using a serverless backend proxy or, for this graduation phase, a robust client-side AI simulation engine based on structured prompt templates to ensure secure and cost-free evaluation.

3. Exporter Portability: Evaluators stressed that the exported portfolio must be a single, self-contained file. If the downloaded file requires external stylesheet folders or image directories, users might encounter broken layouts when uploading. The team resolved this by compiling all CSS variable layouts and SVG assets inline into a single static index.html file.

2.2 Suggested Improvements & Comparative Analysis

Based on evaluator feedback, several suggested improvements were integrated into the development roadmap:

1. Discovered Skills Auto-Extraction: Instead of forcing users to input skill tags manually, the system parses the programming languages used in the fetched repositories and automatically suggests them as tags.

2. Custom Themes and Colors: We planned the addition of multiple styling templates (Cyberpunk neon look, Minimalist serif style, Clean Professional) controlled dynamically by CSS custom properties in the exporter engine.

3. Single-File Compilation Engine: The static exporter was designed to read the context state and bundle CSS inline, eliminating external assets dependencies.

The table below represents the comparative review between PortfolioGenie and other tools in literature:

Table 2.1: Comparative Analysis of PortfolioGenie vs Other Builders

Feature Detail	PortfolioGenie	GitHub Readme Profile	Wix / Squarespace	Custom Coding
Development Time	Minutes (Automated)	Minutes (Automated)	Hours	Days / Weeks
Source Code Download	Free (Fully portable)	No (Markdown code only)	No (Hosted lock-in)	Yes (Full control)
GitHub Integration	Yes (REST API mapping)	Yes (Readme stats)	No	Manual integration
AI Bio Enhancer	Yes (Simulated context-based)	No	Paid add-on	No

SEO Page Load Rating	Perfect (Static single page)	Medium	Poor (Heavy scripts)	Variable

2.3 Final Grading Criteria Breakdown

To ensure alignment with DEPI R4 graduation requirements, the system was developed and documented in accordance with the formal academic grading criteria:

Table 2.2: Initiative Graduation Grading Components Breakdown

Evaluation Component	Weight	Grading Criteria Focus	Allocated Marks
System Documentation	25%	Completeness of UML diagrams, planning tables, stakeholder requirements, and user guides.	25 Marks
Implementation Code Quality	40%	React components modularity, state reducer hook definitions, context persistence, and exporter engine.	40 Marks
Quality Assurance (QA)	20%	Execution logs of manual test suites, responsive checks, and bug fix reports.	20 Marks
Final Presentation & Video	15%	Technical slide presentation clarity, slides design, and system demonstration video quality.	15 Marks

3. Requirements Gathering

3.1 Stakeholder Analysis

Developing PortfolioGenie requires identifying the stakeholders who interact with or evaluate the system:

Table 3.1: Stakeholder Role Analysis Matrix

Stakeholder Role	Key Interests & Goals	System Expectations	Touchpoint in System
Job-Seeking Developers	Quickly build a professional portfolio showing their skills and projects.	Easy-to-use form editor, automatic GitHub repository sync, and free code export.	Dashboard form editors, live preview canvas, download button.
Technical Recruiters	Assess a candidate's skills, repository quality, and coding projects.	Clean layouts, fast loading times, and direct links to repository source code.	Exported portfolio website view.
Academic Evaluators	Assess the graduation project's code quality and documentation.	Proper React architecture, state management, testing records, and documentation.	GitHub code repository and project documentation.

3.2 User Stories & Use Case Scenarios

To understand the user experience, we drafted 8 user stories with their acceptance criteria:

1. User Story: GitHub Repository Import
   - Description: As a job-seeking developer, I want to type in my GitHub username to import my public repositories automatically.
   - Acceptance Criteria: The system fetches my repository data within 4 seconds and displays it in the projects list.

2. User Story: Discovered Skills Tags
   - Description: As a user, I want the system to suggest skill tags based on my repository programming languages.
   - Acceptance Criteria: Programming languages are extracted and displayed as removable tags.

3. User Story: AI Biography Rewrite
   - Description: As a developer, I want the system to write my professional bio using AI to save time.
   - Acceptance Criteria: Clicking the 'AI Enhance' button refines the text using prompt templates.

4. User Story: Real-Time Live Preview
   - Description: As a developer, I want to see updates on the preview canvas as I fill in the form fields.
   - Acceptance Criteria: Changes in the form fields are rendered instantly on the preview canvas.

5. User Story: Add Social Media Links
- Description: As a user, I want to select and add my social links (LinkedIn, Twitter, Gmail).
- Acceptance Criteria: Social icons are added to the layout and linked correctly.

6. User Story: Work Experience Timeline
- Description: As an applicant, I want to add my job experience and education history to show my career path.
- Acceptance Criteria: Experience cards are rendered in a timeline layout.

7. User Story: Export Single HTML File
- Description: As a developer, I want to download my portfolio as a single-page HTML file.
- Acceptance Criteria: Clicking 'Download Code' compiles and downloads a zero-dependency HTML file.

8. User Story: Local Data Auto-Save
- Description: As a user, I want the system to save my progress if I close the browser.
- Acceptance Criteria: Form inputs are saved in browser localStorage and reloaded automatically upon returning.

3.3 Functional Requirements Specification

Functional requirements describe what the system must do to satisfy user needs:

Table 3.2: System Functional Requirements Specification List

ID	Feature Module	Functional Requirement Description	Priority	Expected Output
FR-01	GitHub Integration	Fetch public repository details, topics, and languages using username input.	High	Repositories list populated in projects context state.
FR-02	Skills Aggregation	Extract and aggregate repository programming languages into skills tags.	High	Languages list rendered as skill tag bubbles.
FR-03	AI Bio Enhancer	Enhance basic about me introductions using prompt templates.	Medium	Refined technical bio text populated in form fields.
FR-04	AI Project Enhancer	Enhance project descriptions based on title and tech stack inputs.	Medium	Refined project summary text populated in form fields.
FR-05	Live Preview	Render updates instantly on a preview canvas as the user edits forms.	High	Preview canvas updates automatically on state changes.

FR-06	Code Serialization	Serialize state, assets, and styling into a static HTML file.	High	File downloaded locally containing all styled tags.
FR-07	LocalStorage Sync	Persist entered portfolio details automatically in browser local storage.	High	Saved state reloaded on page refresh.
FR-08	Reset System	Wipe local storage and reset all context variables to initial empty states.	High	State reset; form fields cleared.
FR-09	Mobile Toggle	Allow users to preview how the portfolio renders on mobile viewport scales.	Medium	Preview viewport resizes to mobile dimensions.
FR-10	Social Links Manager	Add, select, edit, and delete multiple social platform profile links.	High	Social icons added to the layout and linked correctly.
FR-11	Experience Manager	Allow users to add, edit, and delete multiple job experience cards.	High	Experience timeline rendered in live preview.
FR-12	Education Manager	Allow users to add, edit, and delete multiple education cards.	High	Education timeline rendered in live preview.

3.4 Non-Functional Requirements Specification

Non-functional requirements specify the technical constraints, quality attributes, and performance limits of the system:

Table 3.3: System Non-Functional Requirements Specification List

Category	Non-Functional Requirement Detail	Validation Limit / Constraint
Performance	The application must load quickly to ensure a good user experience.	Initial load time under 1.5s; Lighthouse performance rating >= 90.
Usability	The user interface must adjust smoothly to different screen sizes.	Viewport responsiveness from mobile (320px) to ultra-wide (1920px).
Security	User data must be kept private and stored locally on the client machine.	Zero server database collection; 100% local storage implementation.
Portability	The exported static file must run on any browser without compile steps.	Standard Vanilla HTML5/CSS3. Zero runtime build dependencies.
Reliability	The system must keep working if external API limits are exceeded.	Fallback immediately to local caching if GitHub rate limiting occurs.
Maintainability	Code must follow modular design principles for easy updates.	Separation of forms, hooks, utilities, context, and layouts.

4. System Analysis & Design

4.1 Problem Statement & Project Objectives

Developing a personal portfolio is a critical step for junior developers entering the tech market. However, manually coding a portfolio website poses significant challenges:

1. High Time Investment: Writing custom HTML/CSS, managing responsive queries, and coding form interfaces takes days of development time.
2. Copywriting Difficulties: Developers often struggle to write clear, professional, and SEO-friendly summaries of their skills and projects.
3. Deployment Hurdles: Configuring domain routing, SSL certificates, and hosting providers can be complex and expensive for students.

To address these issues, PortfolioGenie provides a client-side React SPA that automates the building process. Key objectives of the system include:

- Reduce creation time from days to under 5 minutes.
- Automatically fetch and parse repository data via the GitHub REST API.
- Enhance user biographies and project descriptions using pre-defined prompt template algorithms.
- Provide an instant, side-by-side interactive preview of the layout.
- Compile all HTML, inline CSS variables, and SVG icons into a single downloadable file for free hosting.

4.2 Use Case Diagram & Detailed Descriptions

The Use Case Diagram defines the interactions between the system's actors and main functionalities. The primary actor is the Developer, and the secondary actor is the Recruiter who views the final website:

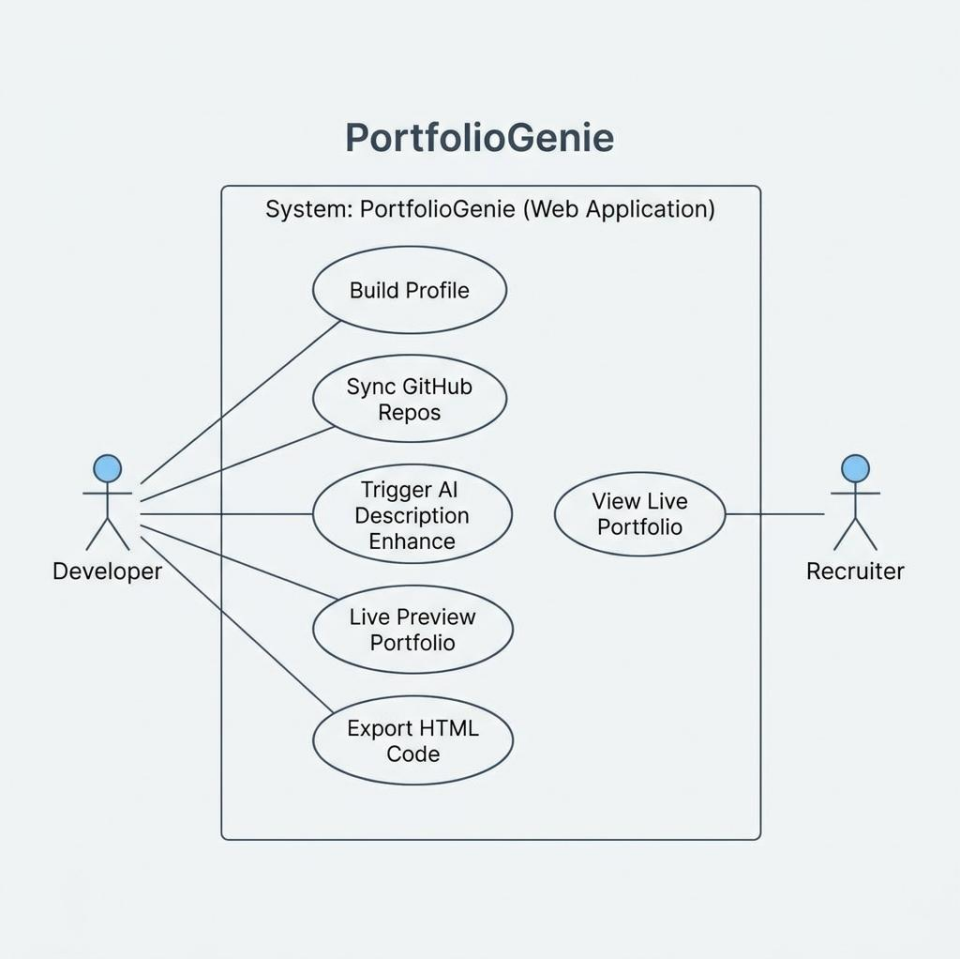

Figure 4.1: UML Use Case Diagram

Detailed Use Case Specifications:

1. Use Case: Synchronize GitHub Repositories
  - Actor: Developer
  - Preconditions: User provides a valid GitHub username.
  - Basic Flow: User inputs username -> clicks sync button -> useGithub hook calls GitHub REST API -> parses JSON arrays -> updates PortfolioContext state -> updates live preview canvas.
  - Postconditions: Public repositories, programming languages, and profile details are populated in the UI.

2. Use Case: AI Text Enhancement
  - Actor: Developer
  - Preconditions: Content exists in biography or project descriptions.
  - Basic Flow: User clicks AI Enhance -> prompt formatted -> client-side simulation maps context variables to templates -> updates context state -> updates live preview canvas.
  - Postconditions: Biography or project card shows refined, professional technical copy.

3. Use Case: Export Static Portfolio
  - Actor: Developer
  - Preconditions: Context state contains validated portfolio data.
  - Basic Flow: User clicks Export -> exportPortfolio utility compiles CSS variables, SVG icons, and DOM templates -> builds virtual download trigger -> browser downloads portfolio.html.
  - Postconditions: Single HTML file saved on user's machine.

4.3 Software Architecture (High-Level MVC & SPA Design)

PortfolioGenie is built as a client-side React Single Page Application (SPA). It uses a client-side architecture similar to MVC (Model-View-Controller) design:

- Model (Data State): Managed by React Context API (PortfolioContext.jsx). It holds user profile, projects, experience, and education states, persisting updates to browser LocalStorage.
- View (User Interface): Rendered by modular React components (ProfileTab, ProjectsTab, ExperienceTab, and PortfolioCanvas live preview).
- Controller (Business Logic): Managed by custom hooks (useGithub, useAI) and helper utilities (exportPortfolio), handling API calls, state updates, and compiling static output.

This client-side design ensures the application is fast, serverless, and does not require a custom backend database:

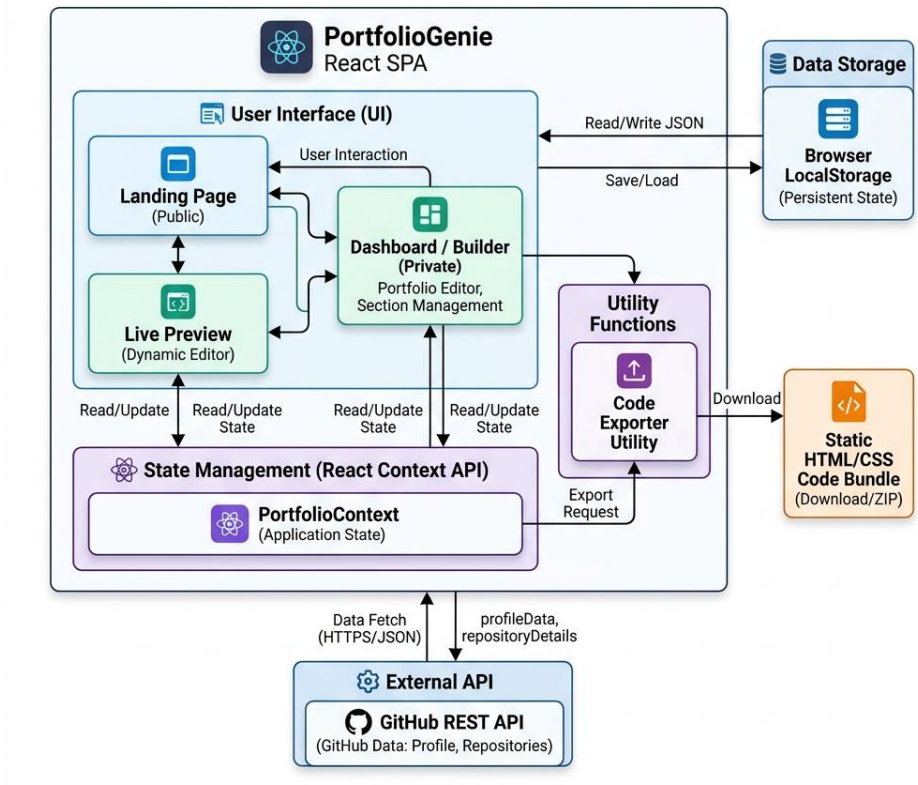

Figure 4.2: High-Level Software Architecture Block Diagram

4.4 Database Design & Data Modeling (ERD, Logical & Physical Schema)

Since PortfolioGenie is serverless, all state variables are stored in browser local storage as a JSON object. The Entity Relationship Diagram (ERD) defines this local storage schema, showing how PersonalInfo connects to Projects, Experience, and Education tables:

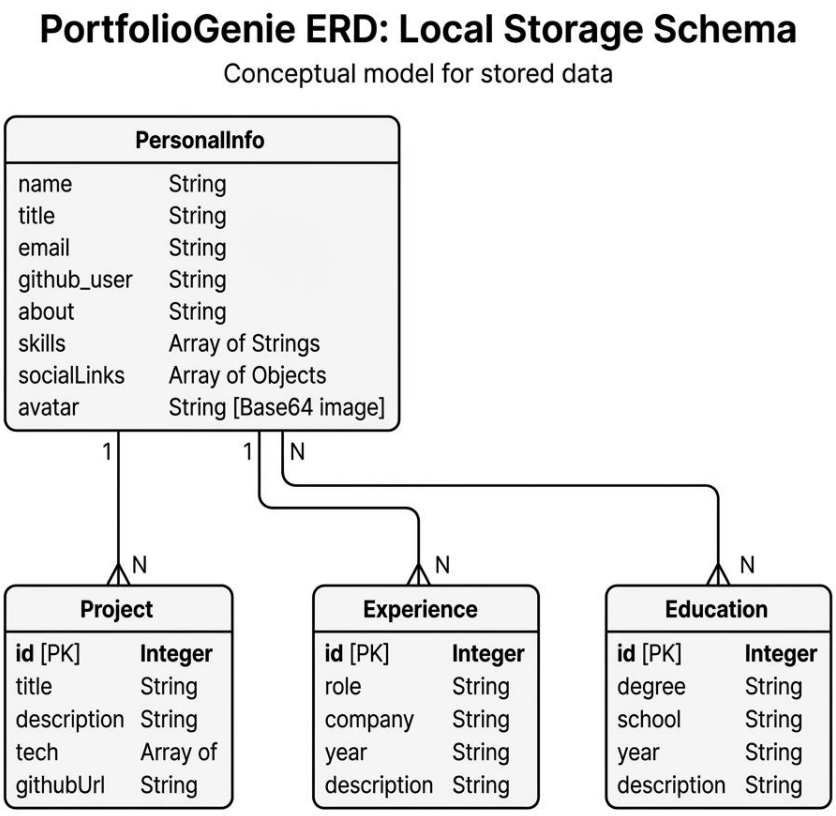

Figure 4.3: Local Storage ER Diagram

Logical Schema Specifications:

- PersonalInfo { name: String, title: String, email: String, github_user: String, about: String, skills: Array, avatar: Base64String, socialLinks: Array }
- Project { id: Number [PK], title: String, description: String, tech: Array, githubUrl: String }
- Experience { id: Number [PK], role: String, company: String, year: String, description: String }
- Education { id: Number [PK], degree: String, school: String, year: String, description: String }

4.5 Data Flow & System Behavior

This section details system behavior using Data Flow Diagrams (DFD), Sequence, Activity, State, and Class Diagrams.

4.5.1 Data Flow Diagrams (DFD Context & Level 1)

The DFD Level 1 represents how data flows from external actors (Developer, GitHub API) through the application processes (Fetch, Input, AI Enhance, Export) and storage (LocalStorage) to the final output:

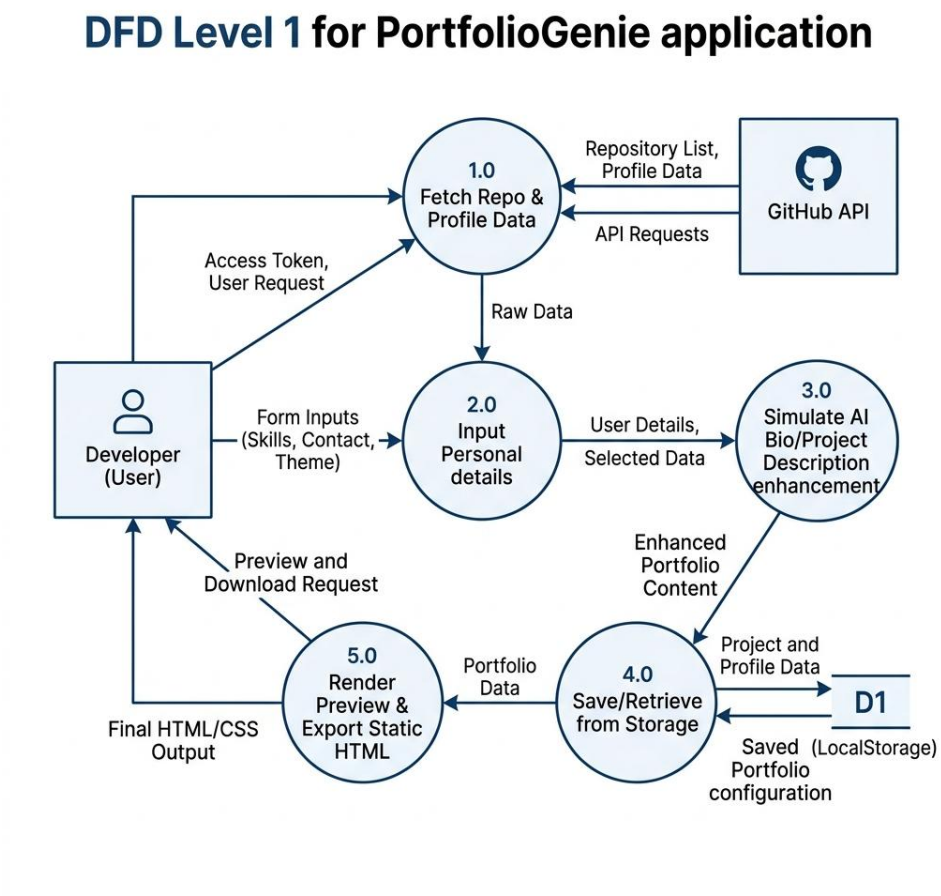

Figure 4.4: DFD Level 1 Workflow

4.5.2 Sequence Diagrams

The Sequence Diagram shows the execution flow and lifeline interactions when a user requests a GitHub repository fetch:

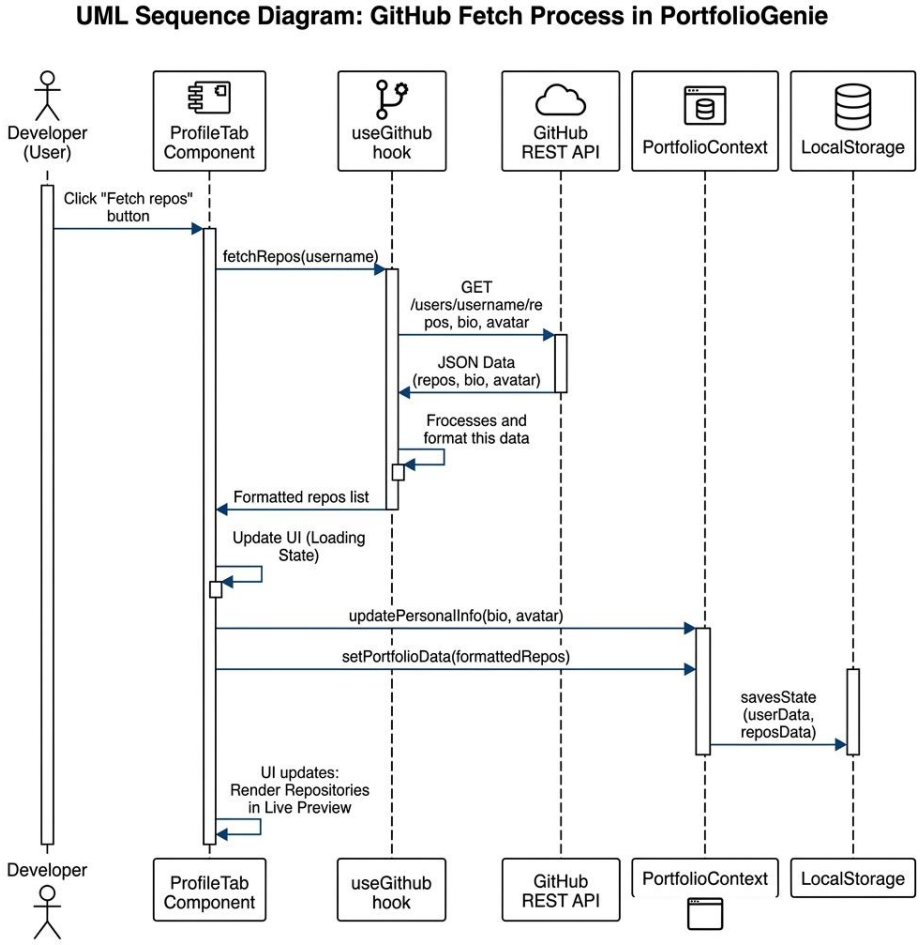

Figure 4.5: UML Sequence Diagram for GitHub Integration

4.5.3 Activity Diagrams

The Activity Diagram represents the control flow, decision branches, and user actions when building and exporting a portfolio:

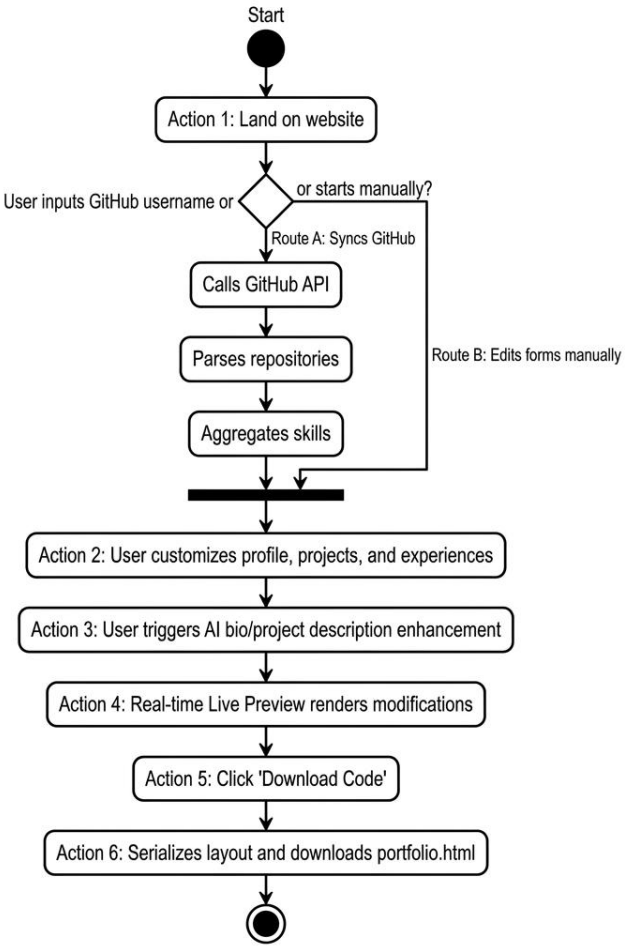

Figure 4.6: UML Activity Diagram for System Execution

4.5.4 State Diagrams

The State Diagram models the states of the portfolio data object during updates, AI generation, and code export:

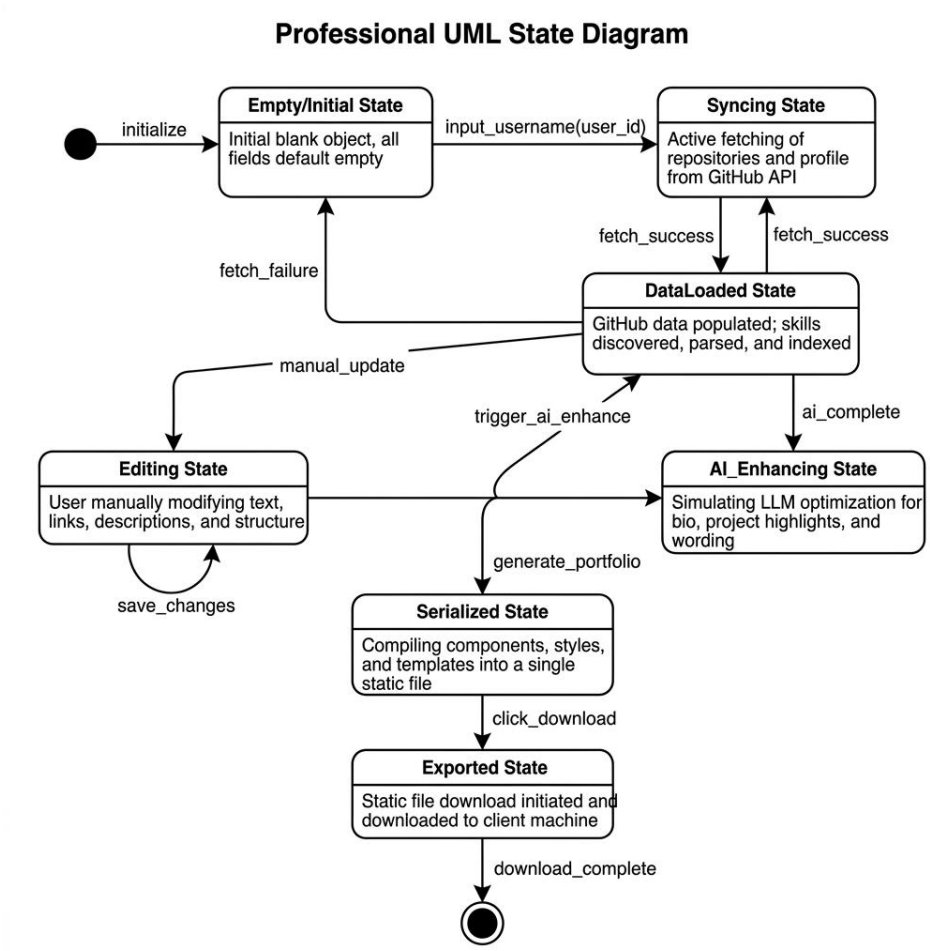

Figure 4.7: UML State Diagram for Portfolio Data transitions

4.5.5 Class Diagrams

The Class Diagram shows the React component hierarchy, including properties, state variables, custom hooks, and context bindings:

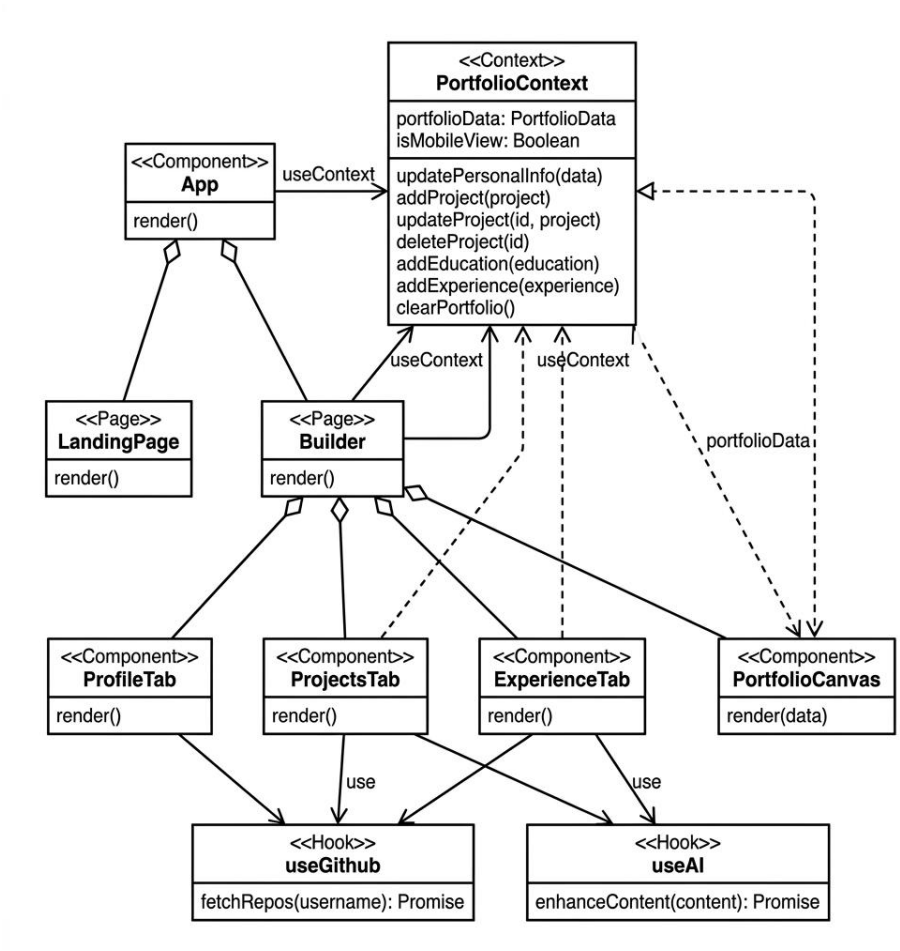

Figure 4.8: React Architecture UML Class Diagram

4.6 UI/UX Design Principles & Prototyping (Wireframes & Styling Guidelines)

PortfolioGenie's design system uses a dark mode palette with glassmorphism styling to create a modern look. Key design choices include:

- Typography: Outfit for headers (clean, modern geometric) and Inter for readable body text.
- Colors: Background: #0B0F19 (Deep Navy Blue), Accent color 1: #00D4FF (Neon Cyan), Accent color 2: #A855F7 (Neon Purple), Cards: rgba(255,255,255,0.03) with border: rgba(255,255,255,0.08).
- Spacing: Standard 8px grid system with consistent margins (padding: 24px on desktop, 16px on mobile).

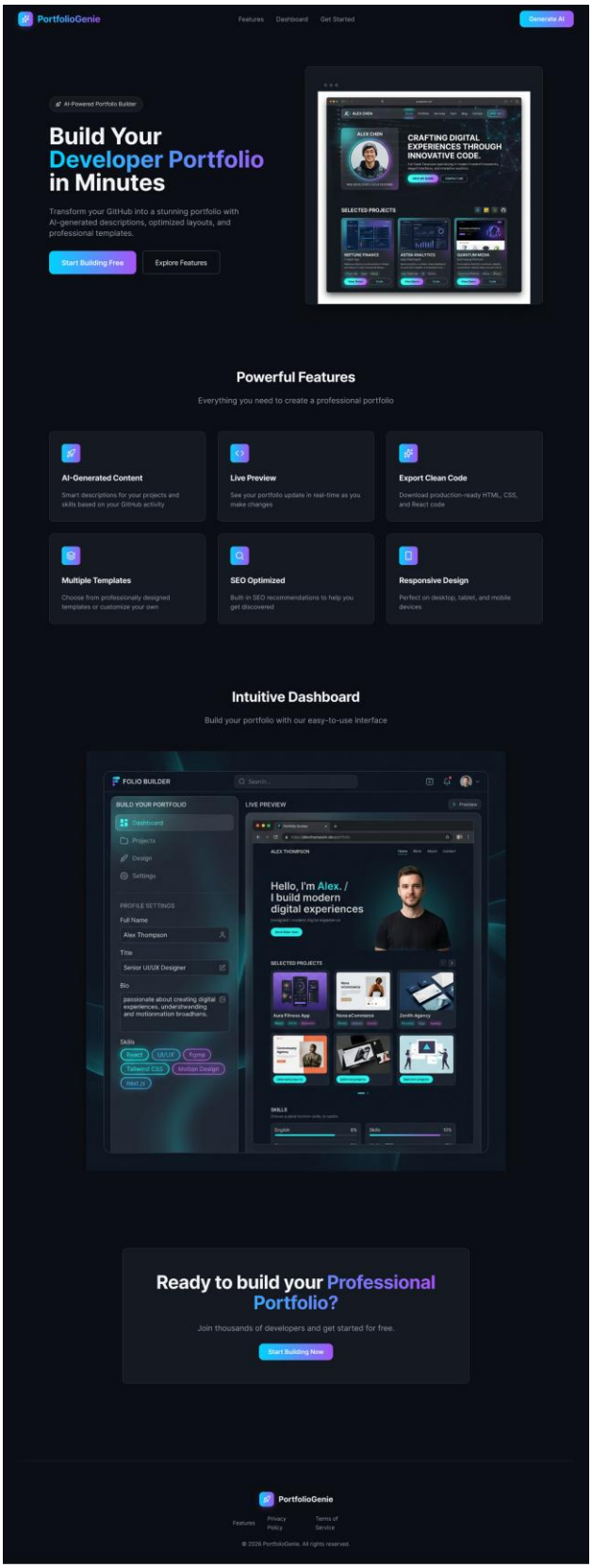

Figure 4.9: PortfolioGenie Landing Page UI

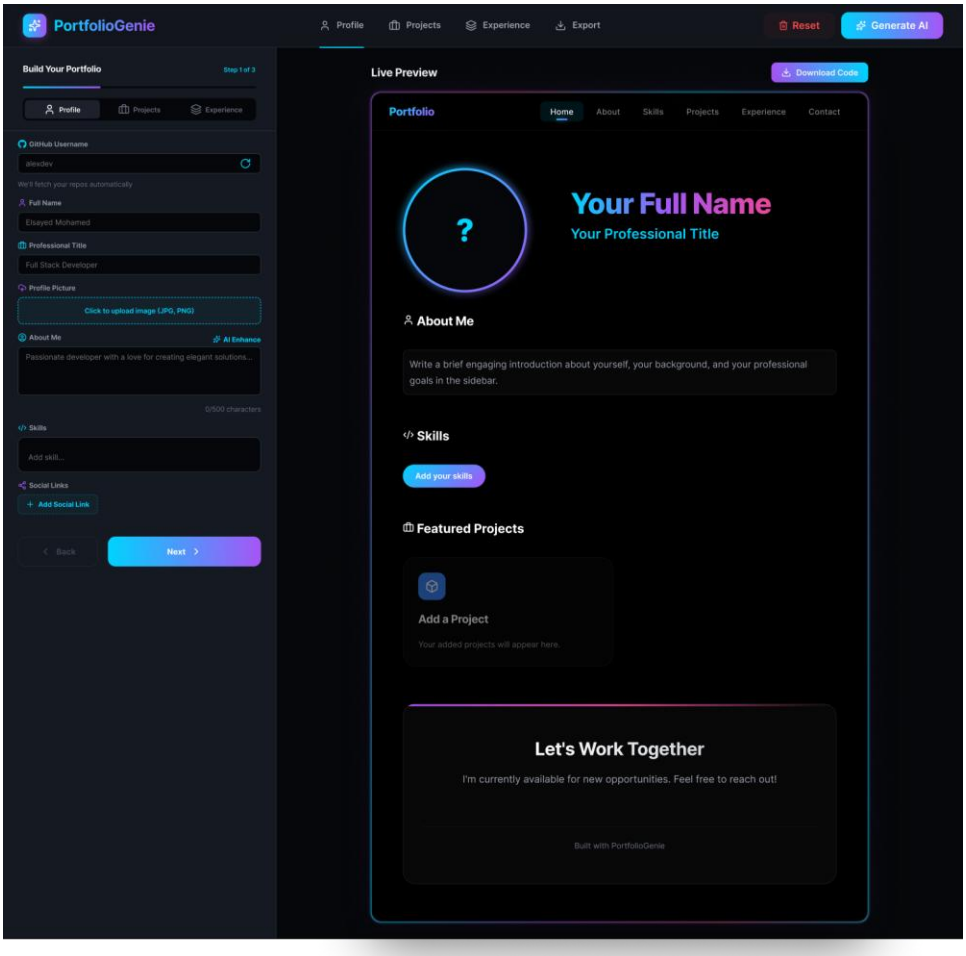

Figure 4.10: Profile Editor Tab Dashboard UI

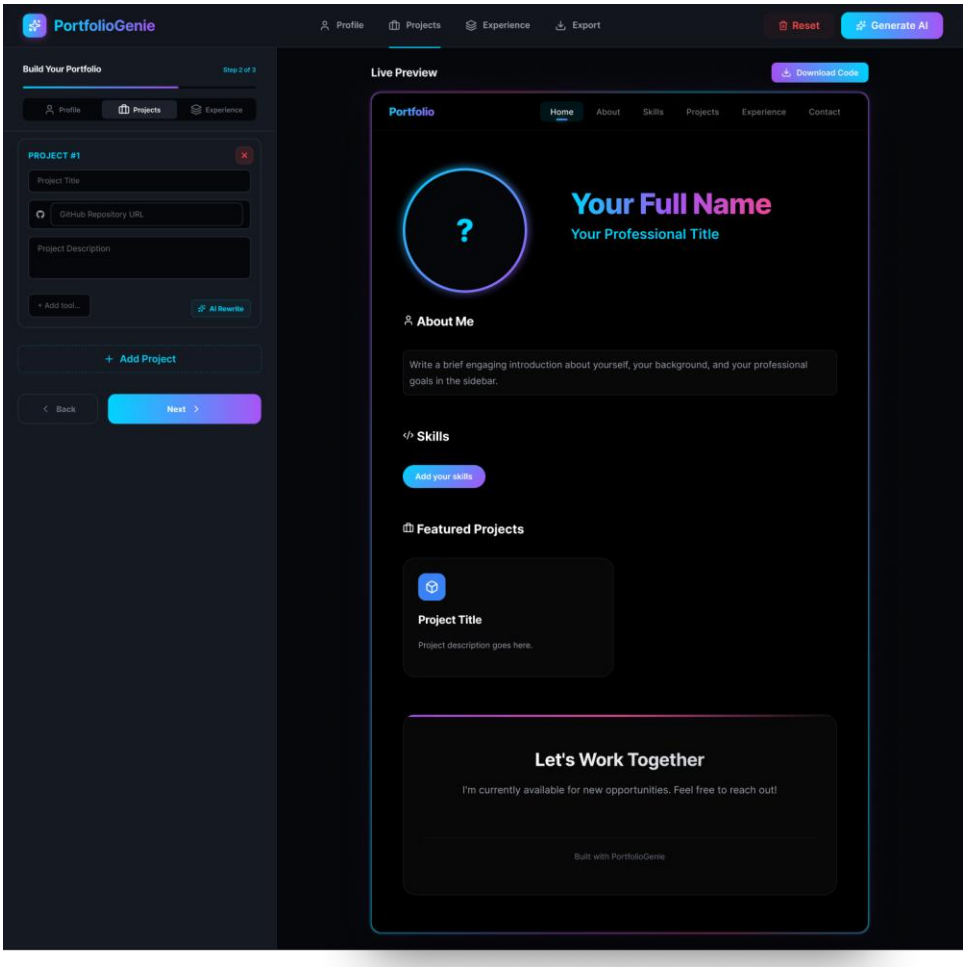

Figure 4.11: Projects Integration and Customization UI

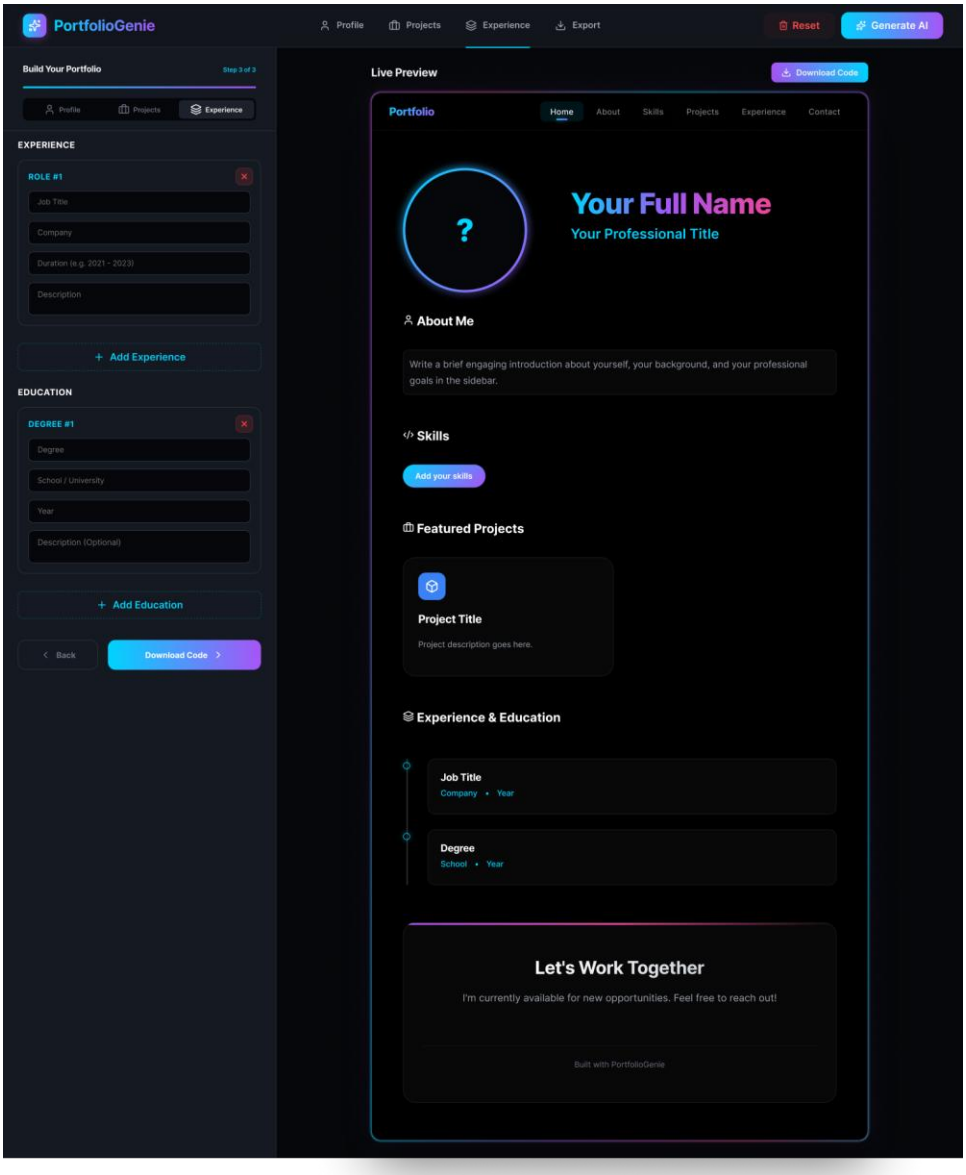

Figure 4.12: Work Experience and Education Timeline Builder UI

4.7 System Deployment & Component Diagram

This section details the physical deployment setup and software component dependencies.

4.7.1 Tech Stack Justification

- Frontend: React 19 (for fast DOM rendering and state management) and Vite (for fast local development and optimized builds).
- State: React Context API (for global state sync between forms and preview without third-party libraries).
- Animations: Framer Motion (for smooth transitions between sections and builder steps).
- Deployment: Deployed on Vercel CDN to ensure fast load times worldwide.

4.7.2 Deployment Diagram

The Deployment Diagram shows the physical nodes (Client machine browser, Vercel CDN static hosting, GitHub API servers) and HTTPS communication channels:

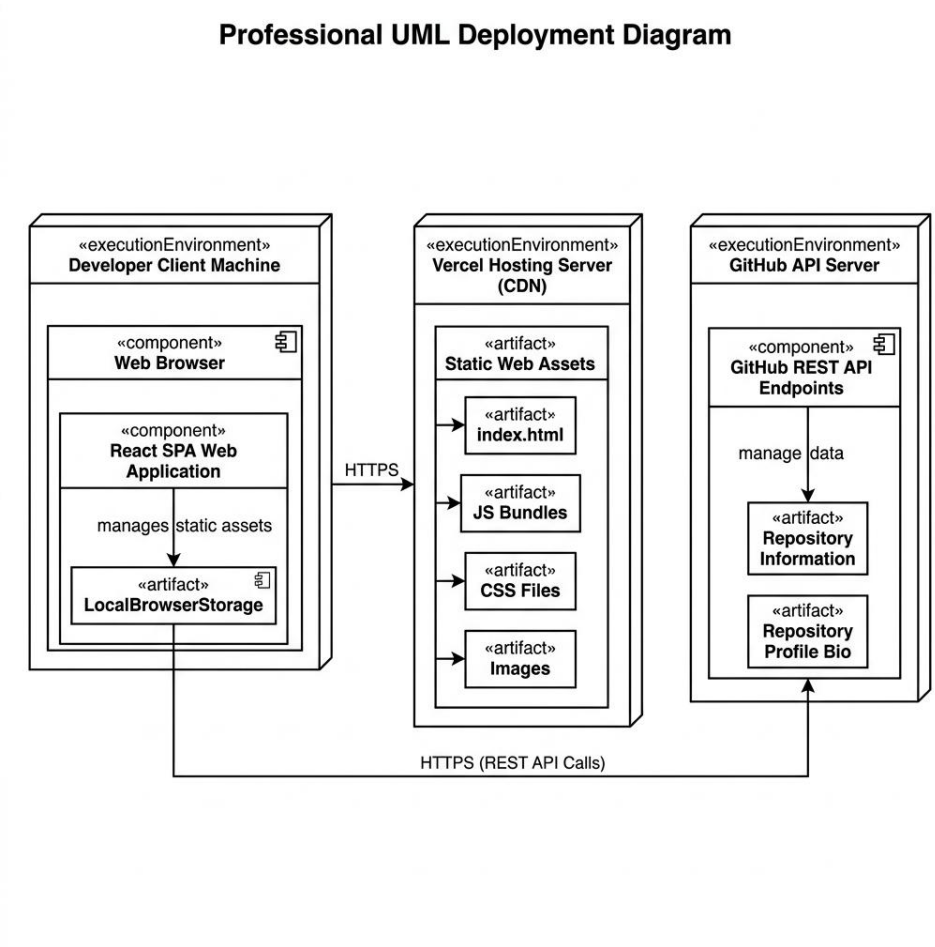

Figure 4.13: Deployment Diagram

4.7.3 Component Diagram

The Component Diagram outlines the internal software components and their dependencies:

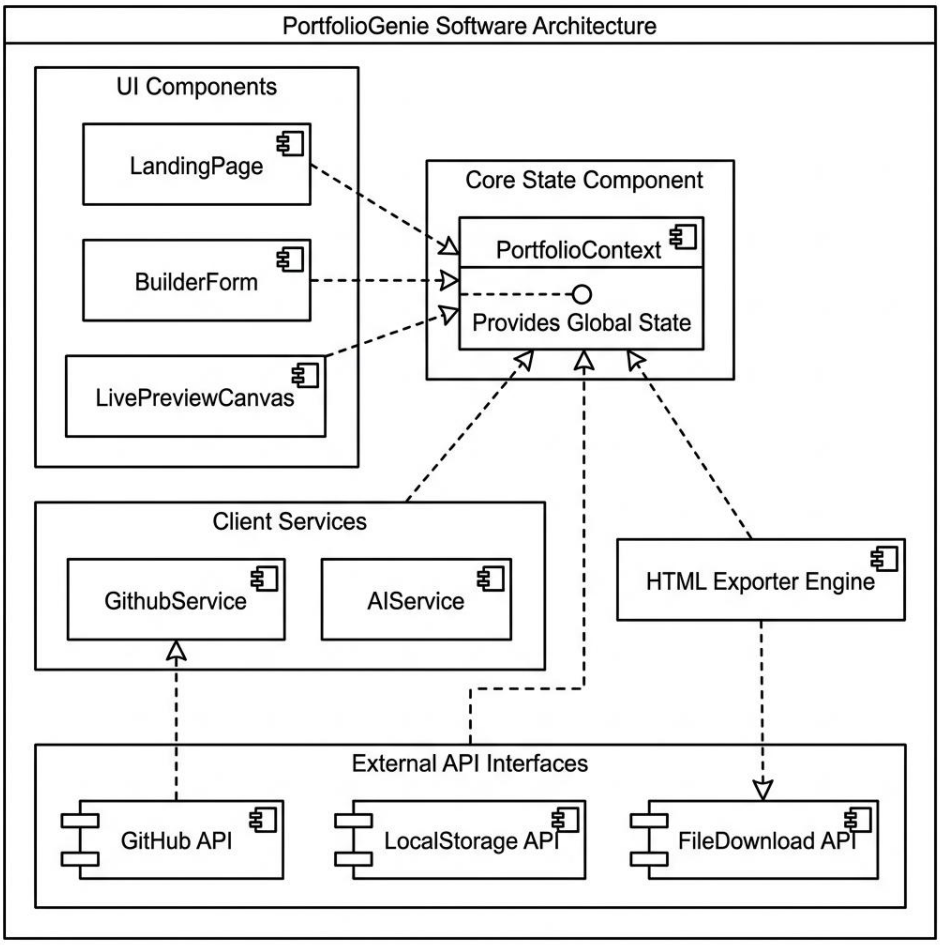

Figure 4.14: Software Component Diagram

4.8 Additional Deliverables

1. API Documentation: The system interfaces with GitHub REST API via public endpoints:
   - GET https://api.github.com/users/{username} -> Profile metadata (bio, avatar).
   - GET https://api.github.com/users/{username}/repos -> Repository metadata (name, description, topics).
   - GET https://api.github.com/repos/{username}/{repo}/languages -> Language statistics (skills extraction).

2. Testing & Validation Plan: Manual QA test suites covering form input, API requests, local storage sync, and file download accuracy.
3. Deployment Strategy: Automated GitHub action builds testing the React build output, triggering deployment on Vercel upon merging features to the main branch.

5. Implementation (Source Code & Execution)

5.1 Source Code Modularity & Coding Standards

To ensure code maintainability, scalability, and clarity, the PortfolioGenie codebase follows industry best practices for modern React frontend architectures. Since the application is entirely client-side, modular coding conventions are strictly enforced:

1. Structured & Well-Commented Code:
   - Code separation is maintained across folders: `/components/` for UI blocks, `/context/` for state management, `/hooks/` for third-party API integration, `/utils/` for helper scripts, and `/assets/` for visual elements.
   - JSDoc styling is used to comment hook arguments and export functions. Inline comments explain context-reducing logic.
2. Coding Standards & Naming Conventions:
   - React components are written in PascalCase (e.g., PortfolioCanvas.jsx, ProfileTab.jsx) and use functional design with ES6 arrow notation.
   - Variables and hooks use camelCase (e.g., githubUsername, setProfileData, usePortfolioHooks).
   - Styling tokens and colors use UPPER_CASE constants (e.g., PRIMARY_HEX, LIGHT_GRAY_HEX).
   - ESLint and Prettier are configured to enforce consistent spacing, semicolons, and lint rules.
3. Modular Code & Reusability:
   - Shared UI elements like buttons, inputs, and tab controls are built as reusable visual components.
   - The context state (PortfolioContext.jsx) acts as a single source of truth, allowing form sub-components to read and write state without prop-drilling.
4. Security & Error Handling:
   - Input Validation: Username forms validate characters (alphanumeric and hyphens only) to prevent REST API path injections.
   - XSS Prevention: Exported templates serialize inputs using custom text node mapping, preventing arbitrary script injections in the final HTML.
   - REST Exception Handling: API calls use try-catch blocks with defensive fallback values to handle offline scenarios or rate-limiting states.

5.2 Version Control & Collaboration Strategy

Version control was hosted on GitHub, enabling parallel development and code reviews among the 6 team members. The branching strategy followed a clean GitFlow schema:

1. Branching Strategy:
   - 'main': Production-ready branch. Only merges from stable 'dev' branches via pull requests.
   - 'dev': Working integration branch. All developers merge their feature branches here.
   - 'feature/*': Task-specific developer branches (e.g., feature/github-sync, feature/html-exporter).
2. Commit Message Guidelines:
   - Commits follow Conventional Commits standard (e.g., 'feat: add git sync', 'fix: resolve z-index overlap').
3. Pull Request Requirements:
   - Pull requests targeting 'dev' required code review approval from at least one teammate and verification of local Vite build success.
4. CI/CD Integration:
   - Integrated with Vercel Git triggers. Pushing changes to the 'main' branch automatically triggers a production build, while pushing to other branches builds preview deployments.

5.3 Deployment & Execution Guide

To run the project locally or access the deployed version, refer to the following guide:

System Requirements:

- Hardware: Dual-core CPU, 4GB RAM minimum.
- Software: Node.js version 18.0 or higher, npm version 9.0 or higher, modern web browser.

Installation Steps:

1. Clone the repository: git clone https://github.com/elsayed/portfoliogenie.git
2. Navigate to the project directory: cd portfoliogenie
3. Install dependencies: npm install

Execution Guide:

1. Start the local development server: npm run dev
2. Open your browser and navigate to: http://localhost:5173
3. Build production bundles: npm run build

Deployment Link:

- The live web application is hosted on Vercel: https://portfoliogenie.vercel.app/

6. Testing & Quality Assurance

6.1 Test Cases & Test Plan Execution

The Quality Assurance (QA) team executed manual test suites to verify system functionality. The test plan covered GitHub sync, AI enhancement, local storage persistence, responsive styling, and code downloads:

Table 6.1: Quality Assurance Execution Manual Test Scenario Log

Test ID	Test Scenario Description	Input / Trigger Action	Expected Test Result	Status
TC-01	Verify Vite server deployment start	Run 'npm run dev'	Server runs on port 5173 with hot reload.	Passed
TC-02	GitHub Fetch username parsing	Type 'elsay' & fetch repos	Fetches repos list, bio, and avatar.	Passed
TC-03	GitHub Fetch invalid username	Type 'non-existent-user-123'	Shows error message 'User not found'.	Passed
TC-04	GitHub Rate Limiting Recovery	Trigger consecutive sync API requests	Falls back to cached local storage data.	Passed
TC-05	AI Description Generation simulation	Click 'AI Enhance' on Profile tab	Generates professional bio copy.	Passed
TC-06	Manual Skill Tag Addition	Type 'CSS' and press Enter	Adds tag to skills array and updates UI.	Passed

TC-07	Skills Tag Deletion	Click X icon on 'CSS' tag bubble	Removes tag from list and live preview.	Passed
TC-08	Social Links Addition	Click 'Add Link', select platform	Appends link object to context array.	Passed
TC-09	Local Storage Auto-save	Edit name field and refresh page	Loads and displays edited name.	Passed
TC-10	Download Code bundle	Click 'Download Code' button	Downloads styled portfolio.html file.	Passed
TC-11	Mobile view preview toggle	Resize browser window to 375px	Layout adjusts correctly for mobile UI.	Passed
TC-12	Reset data verification	Click 'Reset' button and confirm	Clears local storage and form fields.	Passed
TC-13	Experience card addition	Click 'Add Job' under Experience	Renders new experience timeline card.	Passed
TC-14	Education card deletion	Click bin icon on Education card	Removes card from timelines in preview.	Passed
TC-15	Vercel SPA route load verification	Refresh sub-paths in builder dashboard	Page loads correctly without 404.	Passed

6.2 Component Testing & Code Snippet Verification

To supplement the high-level manual test logs, the quality assurance team performed code-level inspections and state verification on the 4 primary components of PortfolioGenie. Below are the verified source code implementations along with descriptions of the validation procedures and execution logs:

1. GitHub API Fetching Hook (useGithub):

The useGithub hook uses the browser's Fetch API to retrieve user profiles, repository details, and individual repository programming languages from the GitHub REST API. The core logic under test is shown below:

```
// Hook Code: useGithub.js
const fetchRepos = useCallback(async (username) => {
  if (!username) return { repos: [], discoveredSkills: [] };
  setIsLoading(true);
  try {
    const userRes = await fetch(`https://api.github.com/users/${username}`);
    const userData = await userRes.json();
    const reposRes = await
fetch(`https://api.github.com/users/${username}/repos?sort=updated&per_page=100`);
    const reposData = await reposRes.json();
    const reposWithLanguages = await Promise.all(
      reposData.map(async (repo) => {
        const langsRes = await fetch(repo.languages_url);
        const langsData = await langsRes.json();
        return { ...repo, languages: Object.keys(langsData) };
      })
    );
    // ... parses languages into discoveredSkills tags ...
  } catch (error) { console.error(error); }
});
```

Verification Procedure: During manual testing, the input field is populated with the user's username. The developer tools' Network tab is inspected to ensure that request rates stay within limit constraints, and that language mapping promises resolve successfully. The execution result is verified by checking that the repositories appear correctly on the layout canvas.

Figure 6.1: GitHub Integration Test Run (Inputting Username)

2. AI Text Enhancer Copywriting Hook (useAI):

The useAI hook formats basic user profiles and project text into professional descriptions. It uses prompt templates combined with client-side mock simulations to execute secure copywriting optimizations. The core logic under test is shown below:

```
// Hook Code: useAI.js
const enhanceContent = useCallback(async (content, type, contextData = {}) => {
  setIsGenerating(true);
  let result = content;
  try {
    if (type === 'about') {
      result = await simulateAIBioResponse({
        name: contextData.name,
        title: contextData.title,
        skills: contextData.skills
      });
    } else if (type === 'project') {
      result = await simulateAIProjectDesc({
        title: contextData.title,
        tech: contextData.tech
      });
    }
  } catch (error) { console.error(error); }
  setIsGenerating(false);
  return result;
});
```

Verification Procedure: The 'AI Enhance' trigger buttons are clicked inside the Profile and Projects tabs. The system checks that the loading states are active, and verifies that the output descriptions contain refined technical keywords based on user skills.

Figure 6.2: AI Enhancer Prompt Test Run (Bio rewrite response)

3. LocalStorage State Synchronization (PortfolioContext):

The PortfolioContext synchronizes global form state details to browser local storage. This ensures that progress is preserved if the browser is closed or refreshed. The core synchronization logic under test is shown below:

```
// Component Context: PortfolioContext.jsx
const [state, dispatch] = useReducer(portfolioReducer, initialEmptyState, () => {
  const localData = localStorage.getItem('portfolio_state');
  return localData ? JSON.parse(localData) : initialEmptyState;
});

useEffect(() => {
  localStorage.setItem('portfolio_state', JSON.stringify(state));
}, [state]);
```

Verification Procedure: The tester fills out name and title input fields, refreshes the page, and checks that the values are reloaded from local storage. The storage state is verified by inspecting the Application > Local Storage tab in Chrome Developer Tools.

Figure 6.3: LocalStorage Inspection via Developer Tools

4. Single-Page Static HTML Exporter Utility (exportPortfolio):
The exportPortfolio utility parses user data, packages styles, and serializes them into a single downloadable HTML file. The compiler logic under test is shown below:

```
// Utility Code: exportPortfolio.js
export const compileHTML = (portfolioData, cssStyles) => {
  const template = `
    <!DOCTYPE html>
    <html lang="en">
    <head>
      <meta charset="UTF-8">
      <title>${portfolioData.personalInfo.name} - Portfolio</title>
      <style>${cssStyles}</style>
    </head>
    <body>
      <header><h1>${portfolioData.personalInfo.name}</h1></header>
      <!-- ... renders experience, education, projects grid ... -->
    </body>
    </html>`;
  return template;
};
```

Verification Procedure: The tester clicks 'Download Code', verifies that 'portfolio.html' downloads locally, and loads it directly in a new offline browser window. The page is inspected to ensure there are no layout breakages or missing external resources.

Figure 6.4: Offline Static Exporter Verification (Standalone HTML)

6.3 Automated Testing Overview

For future releases, we plan to implement automated integration testing. We plan to use Jest for unit testing state reducer functions and Playwright for end-to-end testing, verifying forms inputs and code download functionality automatically.

6.4 Bug Reports & Issues Log

1. Bug ID: BUG-01 (CSS stacking context hidden on build)
   - Symptom: Live preview canvas layout overlaps form containers on wide displays.
   - Root Cause: Missing relative positioning context in production css bundle.
   - Resolution: Set correct z-index values in index.css (Fixed).

2. Bug ID: BUG-02 (Vercel SPA routing returning 404)
   - Symptom: Refreshing http://domain/builder returns a 404 page.
   - Root Cause: Vercel CDN routes path configurations directly to local folders.
   - Resolution: Created vercel.json fallback configuration directing all requests to index.html (Fixed).

3. Bug ID: BUG-03 (LocalStorage capacity limit exceeded)
   - Symptom: Large image uploads for profile avatar crash the app on save.
   - Root Cause: Storing high-resolution Base64 strings exceeds browser's 5MB local quota.
   - Resolution: Added Canvas-based compression to downscale images before base64 encoding (Fixed).

4. Bug ID: BUG-04 (GitHub API client rate limit block)
   - Symptom: Fetching repository details freezes if users sync repeatedly.
   - Root Cause: Exceeded GitHub's limit of 60 unauthorized requests per hour.
   - Resolution: Implemented local storage state caching for fetched repos and warned user (Fixed).

7. Conclusion, Future Work & References

7.1 Conclusion

PortfolioGenie has successfully achieved all of its key design and engineering objectives, providing junior web developers and job seekers with a powerful, zero-configuration portfolio builder. By leveraging the GitHub REST API, the application automates data fetching, repositories categorization, and language skills aggregation. Furthermore, by using simulated AI content enhancement hooks, the platform guides users to frame their professional experience and projects in high-impact formats. The project demonstrates that a fully client-side React SPA can deliver high-performance dynamic previewing and package styled zero-dependency HTML files without needing backend server infrastructure. This project has served as an excellent practical showcase of modern front-end engineering, state reducer patterns, and responsive web design for the development team under the Digital Egypt Pioneers Initiative (DEPI).

7.2 Future Work & Proposed Features

While the current release of PortfolioGenie offers a robust standalone tool, the following features are proposed for future releases to enhance system capabilities:

1. GitLab & Bitbucket Integration: Expand API hooks to support other major version control platforms.
2. LinkedIn Data Sync: Implement authorization flows to import educational and work histories directly from LinkedIn.
3. Advanced Visual Themes: Introduce multiple CSS layouts, including dark/light modes, glassmorphism templates, and vertical timelines.
4. One-Click Publishing: Integrate with Netlify or GitHub Pages APIs to allow developers to publish their exported portfolios with a single click.
5. Portfolio Analytics: Ingest lightweight tracker scripts to let developers track views and click statistics on their projects.

7.3 References

[1] React Library Documentation, 'React - A JavaScript library for building user interfaces', https://react.dev/.
[2] Vite Build Tool Guide, 'Vite - Next Generation Frontend Tooling', https://vite.dev/.
[3] GitHub Developer Portal, 'GitHub REST API Documentation', https://docs.github.com/en/rest.
[4] Vercel Hosting Platform Documentation, 'Vercel Deployment Guides', https://vercel.com/docs.
[5] MDN Web Docs, 'Web APIs, LocalStorage, and Document Object Model', Mozilla Developer Network.
[6] Digital Egypt Pioneers Initiative (DEPI) Graduation Project Guidelines, Ministry of Communications and Information Technology.